

LCQUI

Estratégia e Evidências de Verificação da Implementação

Projeto LCQUI

Baseline de 26 de setembro de 2026

Natureza deste documento. Este documento não constitui fonte normativa do comportamento do LCQUI. A autoridade normativa permanece na especificação principal. Este documento registra estratégia, rastreabilidade e evidências de que a implementação é verificada contra aquela especificação.

O documento principal responde o que o LCQUI deve fazer. Este documento responde como sabemos que a implementação satisfaz a especificação, sem redefinir o comportamento esperado.

Sumário

1	Estratégia	2
1.1	Oráculo e precedência	2
1.2	Camadas	2
1.3	TDD incremental e RED/GREEN	2
1.4	Limitações	2
2	Rastreabilidade	2
2.1	Exemplo não executado	3
3	Backend	3
3.1	Suites existentes	4
4	Security Rules	4
5	E2E	4
6	Resultados da baseline	5
6.1	Estados	5
6.2	Execução de 26 de setembro de 2026	5
6.3	Classificação das 16 falhas de integração	5

1 Estratégia

1.1 Oráculo e precedência

A verificação parte da seguinte precedência:

documentação normativa $\rightarrow$ CUE $\rightarrow$ Alloy $\rightarrow$ matriz de implementação $\rightarrow$ implementação

Os worklogs M0–M13 registram rationale e contraexemplos, mas não substituem a redação normativa final. A implementação existente também não altera o contrato silenciosamente. Um conflito real entre fontes normativas interrompe o trabalho até que seja relatado e resolvido na fonte competente.

1.2 Camadas

- testes unitários isolam funções puras ou dependências controladas por mocks;
- testes de integração exercitam Admin SDK, callables e serviços locais;
- testes de Security Rules usam os Emulators e contextos autenticados;
- testes E2E exercitam fluxos observáveis no navegador;
- build, typecheck e lint fornecem evidência estrutural, não conformidade funcional.

1.3 TDD incremental e RED/GREEN

A unidade de trabalho é uma ou poucas features **IMP-*** fortemente relacionadas. Primeiro se extrai o contrato e se cria um teste que falha pela ausência ou divergência pretendida (RED). A menor mudança de implementação é então feita até o teste passar (GREEN), seguida da regressão relevante. Uma falha de ambiente, dependência ou comando é *INFRA_FAILURE*; ela não é um RED válido de produto.

Uma feature somente pode mudar para **NÃO DIVERGENTE** depois da revisão das obrigações normativas e da aprovação das camadas aplicáveis de backend, Rules, frontend e E2E, sem divergência conhecida restante. Esse estado não equivale a homologação de produção.

1.4 Limitações

Teste aprovado demonstra apenas os cenários cobertos. Mocks não provam integração; compilação não prova invariantes; e uma suite legada pode codificar contrato anterior. Resultados devem registrar versão, comando, escopo e classificação da falha. Ausência de teste não é evidência de conformidade.

2 Rastreabilidade

A cadeia mínima de evidência é:

Norma $\downarrow$ **IMP-*** $\downarrow$ **TEST-*** $\downarrow$ arquivo de teste $\downarrow$ resultado

Cada caso permanente recebe um identificador estável por camada:

- TEST-UNIT-*: lógica isolada;
- TEST-INT-*: integração com serviços locais;
- TEST-RULES-*: autorização em Firestore ou Storage Rules;
- TEST-E2E-*: fluxo observável no navegador.

O sufixo deve ser legível e estável. Quando houver milestone relevante, ele pode ser incorporado, por exemplo TEST-RULES-M9-001. O registro associa o ID à referência normativa exata, à feature da matriz, ao arquivo/símbolo exercitado, ao resultado e à evidência da execução.

2.1 Exemplo não executado

O encadeamento abaixo é somente um exemplo de identificação futura; não declara teste criado nem resultado:

Seção 7, M9 → IMP-RULES-001 → TEST-RULES-M9-001 → futuro arquivo de Rules → resultado ainda não produzido

Não se preenchem resultados futuros e não se cria rastreabilidade retroativa por inferência. A evidência deve nascer da execução registrada.

3 Backend

Os testes de backend são derivados em conjunto de CUE, Alloy, Seções normativas e da divergência descrita na matriz.

Fonte	Tradução para cenário concreto
Fixture CUE válida	cenário positivo com payload, condições e estado posterior esperados
Fixture CUE inválida	rejeição explícita, sem efeitos parciais
Witness Alloy SAT	cenário positivo alcançável, materializado com entidades e relações concretas
Assertion ou propriedade Alloy	um ou mais cenários negativos, de transição ou concorrência
Seção normativa e matriz	autorização, efeitos, falhas, auditoria e lacuna específica da feature

Um resultado UNSAT não é copiado mecanicamente para Jest. A propriedade formal é traduzida em arranjo, operação, interleaving quando aplicável e assertions observáveis. Jest é usado para lógica/callables; Firebase Emulator Suite, para integração; e @firebase/rules-unit-testing, para Rules.

3.1 Suites existentes

Na baseline desta rodada, `auth.test.ts` e `domain/*.test.ts` são unitários/mocados. Os testes de integração sob `integration/` e os testes de notificações, patrimônio, reagentes, relatórios, roteiros e turmas usam Admin SDK ou `firebase-functions-test` em processo e dependem dos Emulators. Nenhum deles chama o Functions Emulator por HTTP.

Os 46 casos atuais de `relatorios.test.ts` contêm assertions triviais e são classificados como testes estruturais/textuais de baixo valor. Permanecem na baseline, mas um PASS nesses casos não prova os comportamentos descritos.

4 Security Rules

Security Rules são verificadas com `@firebase/rules-unit-testing` contra Firestore e Storage Emulators locais. Casos devem cobrir, conforme aplicável:

- acesso permitido e acesso negado;
- o mesmo ato por outro usuário;
- o mesmo usuário sobre outro recurso;
- token ou claims obsoletos;
- conta inativa, versão divergente e papel persistido divergente;
- documento de autoridade ausente e estado alterado entre leitura e commit.

Dados de preparação são inseridos com Rules desabilitadas somente no bloco de arranjo. A operação sob teste usa contexto autenticado ou não autenticado real. Nenhum teste desta camada pode atingir um projeto Firebase remoto.

A primeira feature planejada para este pipeline é IMP-RULES-001, sobre autoridade de usuário, versão e papéis. Ela não foi implementada nem corrigida nesta rodada.

5 E2E

Os casos E2E derivam principalmente da Seção 9 (fluxos), da Seção 8 (telas e estados) e da Seção 7 (resultados e regras). O estado atual da interface não é fonte para inventar comportamento.

Playwright CLI é ferramenta de exploração: snapshots, locators, requests, console, screenshots e depuração. Playwright Test mantém specs, assertions e regressão automatizada. A exploração não substitui a suite permanente.

Locators permanentes preferem papel, label e texto semanticamente estável; `data-testid` é usado quando necessário. Seletores CSS baseados na estrutura visual devem ser evitados. Sessões exploratórias independentes não compartilham estado sem intenção explícita.

O bootstrap Playwright 1.61.1 com browsers Nix está disponível, mas esta rodada não criou configuração de suite nem teste E2E de negócio. Esses artefatos devem nascer com o primeiro fluxo real.

6 Resultados da baseline

6.1 Estados

PASS comando executou e todas as verificações abrangidas passaram;

FAIL comando executou e preservou uma ou mais falhas;

NOT_RUN não executado nesta baseline;

BLOCKED pré-condição externa impediu a execução;

INFRA_FAILURE ferramenta, dependência ou ambiente impediu observar o produto;

KNOWN_IMPLEMENTATION_DIVERGENCE falha coerente com divergência já registrada na matriz;

LEGACY_TEST_EXPECTATION fixture ou expectativa codifica interface/estado anterior e requer rederivação normativa.

6.2 Execução de 26 de setembro de 2026

Ambiente observado: Node 26.8.1, npm 11.19.0, Java 21.0.12, Jest 29.7.0, Firebase CLI 15.29.0 e Playwright 1.61.1. O backend declara Node 20; o aviso de engine produzido pelo Node 26 fica registrado e não impediu a baseline.

Verificação	Estado	Evidência
Backend build	PASS	<code>npm run build</code> ; exit 0. <code>functions/lib</code> permaneceu ignorado.
Backend lint	FAIL	<code>npm run lint</code> ; ESLint local iniciou e encontrou 82 erros existentes, sem falha de infraestrutura.
Jest unitário	PASS	<code>npm run test:unit</code> ; 3 suites, 53 testes aprovados, 0 falhas.
Rules	PASS	<code>npm run test:rules</code> ; 2 suites, 48 testes aprovados, 0 falhas; Emulators encerrados.
Integração	FAIL	<code>npm run test:integration</code> ; 7 suites: 3 passaram e 4 falharam; 69 testes passaram e 16 falharam.
Wrapper agregado	FAIL	<code>npm run test:emulator</code> ; Rules passaram, integração preservou exit 1 e os Emulators foram encerrados.
Frontend typecheck	PASS	<code>npx -no-install tsc -noEmit</code> ; exit 0.
Frontend lint	FAIL	<code>npm run lint</code> ; 79 erros e 29 avisos existentes.
Playwright boots-trap	PASS	Dependências e browser Nix 1.61.1; smoke aprovado na rodada anterior, não repetido.

6.3 Classificação das 16 falhas de integração

Treze falhas são classificadas como **LEGACY_TEST_EXPECTATION** nesta baseline: dez fixtures de reagentes não fornecem `idResumoReagente`; uma expectativa patrimonial exige texto antigo de validação; e dois casos de usuário não preparam a autoridade ativa agora exigida antes da regra que pretendem testar. Essa classificação não autoriza atualizar os testes sem rederivá-los da norma.

Três falhas são `KNOWN_IMPLEMENTATION_DIVERGENCE`: uma em patrimônio e duas em compartilhamento de roteiro executam leitura depois de escrita na mesma transação Firestore. As áreas correspondentes já estão `DIVERGENTES` na matriz. Nenhuma causa foi corrigida nesta rodada.

Não houve `INFRA_FAILURE` após a instalação local do ESLint e a criação dos wrappers. A baseline vermelha representa código/expectativas existentes e continua visível pelos exit codes.